



Student: *Konstantinos Kritsotakis*
Registration Number: *7115152500007*
Background: *Chemical Engineer, National Technical University of Athens (NTUA)*
Graduate Student: *MSc DSIT – Specialization: Big Data & Artificial Intelligence*

Methodology

The preprocessing steps were:

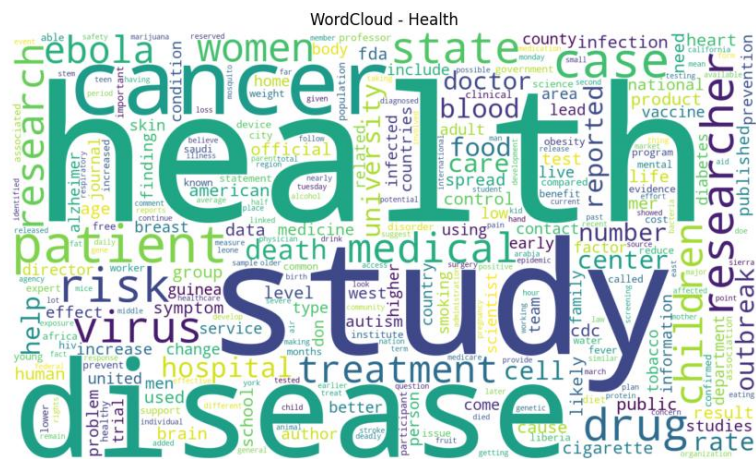
- The WordCloud size of each word is proportional to its frequency within the specific category corpus.

[illegible]

Entertainment

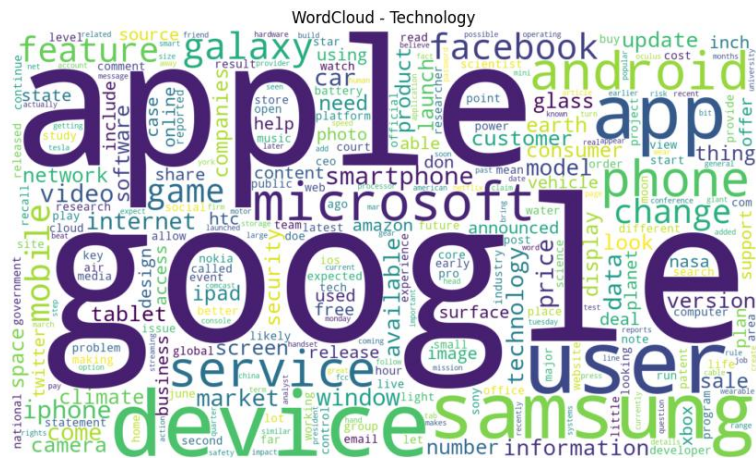
Frequent terms such as film, movie, star, season, music, actor, award, series highlight content related to cinema, television, celebrities, and the entertainment industry. The vocabulary reflects media coverage and popular culture topics.

Health



Key words include cases, virus, disease, drug, medical, patient, hospital, research, indicating coverage of public health issues, medical research, infectious diseases, and clinical topics. The category clearly centers around healthcare and epidemiology.

Technology



Prominent terms such as google, microsoft, app, phone, service, device, software, technology show a focus on major tech companies, digital platforms, consumer electronics, and software innovation. The vocabulary reflects developments in the technology sector.

Appendix - Code Used to Generate WordClouds

```
import pandas as pd
import re
from sklearn.feature_extraction.text import ENGLISH_STOP_WORDS
from wordcloud import WordCloud
import matplotlib.pyplot as plt

df = pd.read_excel("train.xlsx")

BASE_STOPWORDS = set(ENGLISH_STOP_WORDS)
EXTRA_STOPWORDS = {
    "said", "says", "say", "also", "one", "two", "new", "year", "years",
    "week", "weeks", "day", "days", "time", "times",
    "mr", "mrs", "ms", "u", "us", "uk", "eu",
    "reuters", "ap", "cnn", "bbc", "news",
    "didn", "doesn", "isn", "aren", "wasn", "weren", "hasn", "haven", "hadn",
    "won", "wouldn", "couldn", "shouldn", "mustn", "mightn",
    "im", "ive", "youre", "theyre", "weve", "youve",
    "like", "just", "got", "get", "go", "going"
}
STOPWORDS = BASE_STOPWORDS | EXTRA_STOPWORDS | {
    "people", "company", "million", "percent", "world", "according", "make", "billion",
    "way", "know", "good", "report", "high", "think", "including", "long", "best", "use",
    "today", "month", "april", "old", "really", "big", "work", "end", "based", "set", "want",
    "right", "told", "did"
}

def clean_text(s):
    if not isinstance(s, str):
        return ""
    s = s.lower()
    s = re.sub(r"http\S+|www.\S+|\S+@\S+|\d+", " ", s)
    s = re.sub(r"[^a-z\s]", " ", s)
    return re.sub(r"\s+", " ", s).strip()

def tokenize filter(s):
```

```

        return " ".join([w for w in s.split() if w not in STOPWORDS and len(w) >= 3])

df["text"] = (df["Title"].fillna("") + " " +
df["Content"].fillna("")).map(clean_text)
df["text"] = df["text"].map(tokenize_filter)

for lab in ["Business", "Entertainment", "Health", "Technology"]:
    corpus = " ".join(df[df["Label"]==lab]["text"])
    wc = WordCloud(width=1400,height=800,background_color="white",
                    max_words=300,collocations=False,
                    random_state=42).generate(corpus)
    wc.to_file(f"wordcloud_{lab.lower()}.png")

```

Question 1.2 (Classification Task)

Evaluation Results

Statistic Measure	SVM (BoW)	KNN with Jaccard	My Method
Accuracy (5-fold CV)	0.96477	0.96821	0.97537

We evaluated two baseline classifiers using Bag of Words (BoW) features and 5-fold Stratified Cross-Validation with Accuracy. The BoW + Linear SVM baseline achieved 0.96477 mean accuracy, while KNN with Jaccard similarity (binary BoW) achieved 0.96821. To beat the benchmark, we experimented with alternative feature weighting and preprocessing and used **TF-IDF** (unigrams + bigrams, sublinear TF) with a tuned **Linear SVM**, which improved performance to **0.97537** mean accuracy. This indicates that **TF-IDF weighting and sublinear term** frequency reduce the impact of very frequent terms and improve separability for linear margins.

Beat the Benchmark

- **Feature change:** BoW → **TF-IDF** to down-weight common terms and emphasize informative terms.
- **Sublinear TF:** `sublinear_tf=True` applies **1 + log(tf)**, limiting dominance of repeated words.
- **Context features:** `ngram_range=(1,2)` captures informative bigrams (e.g., “stock market”, “health care”).
- **Model tuning:** LinearSVC with `C=1.5` and `max_iter=5000` for stable optimization and improved accuracy.

Finally, we trained **the best-performing model (TF-IDF + Linear SVM)** on the full train set and generated the required submission file `testSet_categories.csv` with columns `Id, Predicted` (comma-separated) for all test instances, which was uploaded to Kaggle.

Appendix A - Source Code (Classification Task)

The following code implements the preprocessing pipeline, baseline models (SVM with BoW, KNN with Jaccard), and the proposed method (TF-IDF + Linear SVM), evaluated using 5-fold Stratified Cross-Validation with Accuracy on the train set.

```

# =====
# Q1.2 - FULL SCRIPT (Baseline + My Method)
# =====

import pandas as pd
import numpy as np
import re

```

```

import time

from sklearn.model_selection import StratifiedKFold
from sklearn.pipeline import Pipeline
from sklearn.feature_extraction.text import ENGLISH_STOP_WORDS
from sklearn.feature_extraction.text import CountVectorizer, TfidfVectorizer
from sklearn.svm import LinearSVC
from sklearn.metrics import accuracy_score
from collections import Counter
from scipy import sparse

# -----
# 0) Load train set
# -----
TRAIN_PATH = r"train.xlsx"

df = pd.read_excel(TRAIN_PATH)
y = df["Label"].astype(str).values

# -----
# 1) Preprocessing
# -----
BASE_STOPWORDS = set(ENGLISH_STOP_WORDS)
STOPWORDS = BASE_STOPWORDS | {
    "said", "says", "also", "new", "year", "week", "day", "mr", "mrs",
    "reuters", "cnn", "bbc", "news", "people", "company", "million",
    "percent", "world", "according", "make", "billion", "today"
}

def clean_text(s):
    if not isinstance(s, str):
        return ""
    s = s.lower()
    s = re.sub(r"http\S+|www\.\S+|\S+@\S+|\d+", " ", s)
    s = re.sub(r"^[a-z\s]", " ", s)
    return re.sub(r"\s+", " ", s).strip()

def tokenize_filter(s):
    return " ".join([w for w in s.split() if w not in STOPWORDS and len(w) >= 3])

X_raw = (df["Title"].fillna("") + " " + df["Content"].fillna("")).values
X_text = np.array([tokenize_filter(clean_text(t)) for t in X_raw], dtype=object)

# -----
# 2) CV helper
# -----
def cv_accuracy(X, y, estimator, n_splits=5, seed=42):
    skf = StratifiedKFold(n_splits=n_splits, shuffle=True, random_state=seed)
    accs = []
    t0 = time.time()

    for fold, (tr, va) in enumerate(skf.split(X, y), 1):
        est = estimator
        est.fit(X[tr], y[tr])
        pred = est.predict(X[va])
        acc = accuracy_score(y[va], pred)
        accs.append(acc)
        print(f"[Fold {fold}] Accuracy = {acc:.5f}")

    dt = time.time() - t0
    print(f"\nMean Accuracy = {np.mean(accs):.5f} | Std = {np.std(accs):.5f} | Time = {dt/60:.1f} min\n")
    return np.array(accs)

# =====
# BASELINE A: BoW + Linear SVM

```



```

# =====
svm_bow = Pipeline([
    ("bow", CountVectorizer(
        lowercase=False,
        tokenizer=str.split,
        preprocessor=None,
        token_pattern=None,
        ngram_range=(1,2),
        min_df=2,
        max_df=0.95,
        max_features=200_000
    )),
    ("clf", LinearSVC(C=1.0, max_iter=5000, random_state=42))
])

print("\n=====")
print("Baseline: SVM (BoW)")
print("=====")
svm_accs = cv_accuracy(X_text, y, svm_bow)

# =====
# BASELINE B: KNN + Jaccard
# =====
def fit_bow_binary(X_text, max_features=100_000):
    vec = CountVectorizer(
        lowercase=False,
        tokenizer=str.split,
        preprocessor=None,
        token_pattern=None,
        binary=True,
        ngram_range=(1,1),
        min_df=2,
        max_df=0.95,
        max_features=max_features
    )
    return vec, vec.fit_transform(X_text).astype(np.bool_)

def transform_bow_binary(vec, X_text):
    return vec.transform(X_text).astype(np.bool_)

def jaccard_knn(X_train_bin, y_train, X_query_bin, k=5):
    maj = Counter(y_train).most_common(1)[0][0]
    train_nnz = np.asarray(X_train_bin.getnnz(axis=1)).reshape(-1)
    preds = []

    inter = (X_query_bin.astype(np.uint8) @ X_train_bin.T.astype(np.uint8)).tocsr()
    q_nnz = np.asarray(X_query_bin.getnnz(axis=1)).reshape(-1)

    for i in range(inter.shape[0]):
        row = inter.getrow(i)
        if row.nnz == 0:
            preds.append(maj)
            continue

        idx = row.indices
        inter_counts = row.data.astype(np.float32)
        union = q_nnz[i] + train_nnz[idx] - inter_counts
        jac = inter_counts / np.maximum(union, 1.0)

        topk = idx[np.argpartition(-jac, min(k, len(jac))-1)[:k]]
        preds.append(Counter(y_train[topk]).most_common(1)[0][0])

    return np.array(preds)

def cv_knn(X_text, y):
    skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

```

```

accs = []

for fold, (tr, va) in enumerate(skf.split(X_text, y), 1):
    vec, Xtr = fit_bow_binary(X_text[tr])
    Xva = transform_bow_binary(vec, X_text[va])
    pred = jaccard_knn(Xtr, y[tr], Xva)
    acc = accuracy_score(y[va], pred)
    accs.append(acc)
    print(f"[Fold {fold}] Accuracy = {acc:.5f}")

print(f"\nMean Accuracy = {np.mean(accs):.5f} | Std = {np.std(accs):.5f}\n")
return np.array(accs)

print("\n=====")
print("Baseline: KNN (Jaccard)")
print("=====")
knn_accs = cv_knn(X_text, y)

# =====
# MY METHOD: TF-IDF + Optimized Linear SVM
# =====
my_model = Pipeline([
    ("tfidf", TfidfVectorizer(
        lowercase=False,
        tokenizer=str.split,
        preprocessor=None,
        token_pattern=None,
        ngram_range=(1,2),
        min_df=2,
        max_df=0.9,
        max_features=300_000,
        sublinear_tf=True
    )),
    ("clf", LinearSVC(
        C=1.5,
        max_iter=5000,
        random_state=42
    ))
])

print("\n=====")
print("MY METHOD: TF-IDF + Linear SVM")
print("=====")
my_accs = cv_accuracy(X_text, y, my_model)

# =====
# FINAL COMPARISON
# =====
print("\n===== FINAL RESULTS =====")
print(f"SVM (Bow):      {svm_accs.mean():.5f}")
print(f"KNN (Jaccard):   {knn_accs.mean():.5f}")
print(f"MY METHOD:        {my_accs.mean():.5f}")

```

Appendix B - Source Code (Classification Task)

The following code trains the best-performing model (TF-IDF + Linear SVM) on the full training set and generates the required submission file testSet_categories.csv containing the predictions for the test dataset.

```

import pandas as pd
import numpy as np
import re
from sklearn.feature_extraction.text import ENGLISH_STOP_WORDS, TfidfVectorizer
from sklearn.svm import LinearSVC
from sklearn.pipeline import Pipeline

```

```

# -----
# 0) Paths
# -----
TRAIN_PATH = r"train.xlsx"
TEST_PATH  = r"test_without_labels.xlsx"
OUT_PATH   = r"testSet_categories.csv"

# -----
# 1) Load
# -----
train_df = pd.read_excel(TRAIN_PATH)
test_df  = pd.read_excel(TEST_PATH)

y_train = train_df["Label"].astype(str).values

# -----
# 2) Preprocessing (SAME as CV)
# -----
BASE_STOPWORDS = set(ENGLISH_STOP_WORDS)
EXTRA_STOPWORDS = {
    "said", "says", "say", "also", "one", "two", "new", "year", "years",
    "week", "weeks", "day", "days", "time", "times",
    "mr", "mrs", "ms", "u", "us", "uk", "eu",
    "reuters", "ap", "cnn", "bbc", "news",
    "didn", "doesn", "isn", "aren", "wasn", "weren", "hasn", "haven", "hadn",
    "won", "wouldn", "couldn", "shouldn", "mustn", "mightn",
    "im", "ive", "youre", "theyre", "weve", "youve",
    "like", "just", "got", "get", "go", "going"
}
STOPWORDS = BASE_STOPWORDS | EXTRA_STOPWORDS | {
    "people", "company", "million", "percent", "world", "according", "make", "billion",
    "way", "know", "good", "report", "high", "think", "including", "long", "best", "use",
    "today", "month", "april", "old", "really", "big", "work", "end", "based", "set", "want",
    "right", "told", "did"
}

def clean_text(s):
    if not isinstance(s, str):
        return ""
    s = s.lower()
    s = re.sub(r"http\S+|www.\S+|\S+@\S+|\d+", " ", s)
    s = re.sub(r"^[a-z\s]", " ", s)
    return re.sub(r"\s+", " ", s).strip()

def tokenize_filter(s):
    return " ".join([w for w in s.split() if w not in STOPWORDS and len(w) >= 3])

def build_text(df):
    raw = (df["Title"].fillna("") + " " + df["Content"].fillna("")).values
    return np.array([tokenize_filter(clean_text(t)) for t in raw], dtype=object)

X_train_text = build_text(train_df)
X_test_text  = build_text(test_df)

# -----
# 3) Define MY METHOD model
# -----
model = Pipeline([
    ("tfidf", TfidfVectorizer(
        lowercase=False,
        tokenizer=str.split,
        preprocessor=None,
        token_pattern=None,
        ngram_range=(1,2),
        min_df=2,
        max_df=0.9,

```



```

        max_features=300_000,
        sublinear_tf=True
    )),
    ("clf", LinearSVC(
        C=1.5,
        max_iter=5000,
        random_state=42
    ))
])

# -----
# 4) Train on FULL train
# -----
model.fit(X_train_text, y_train)

# -----
# 5) Predict FULL test
# -----
test_pred = model.predict(X_test_text)

# -----
# 6) Export
# -----
out = pd.DataFrame({
    "Id": test_df["Id"].astype(int),
    "Predicted": test_pred
})

out.to_csv(OUT_PATH, index=False)
print(f"Saved: {OUT_PATH} | rows={len(out)}")
print(out.head())

```

Question 2 - Nearest Neighbor Search with and without Locality Sensitive Hashing (LSH)

Brute-Force Baseline

The brute-force approach computes the exact Jaccard similarity between every test document and every training document.

Time complexity:

$$O(|Test| \cdot |Train|)$$

Properties:

- Exact similarity computation
- Recall = 1.0 (by definition)
- Very high query time

This serves as the gold standard reference. Also, it is good to clarify that metric *Recall* is computed as:

$$\text{Recall} = \frac{\text{LSH top-K} \cap \text{brute top-K}}{K}$$

LSH with Explicit Threshold = 0.9 (As Required)

In this first implementation:

<code>MinHashLSH(num_perm=num_perm, threshold=0.9)</code>

The library automatically selects (b,r) such that the S-curve knee is near 0.9.

Thus:

$$P(s) = 1 - (1 - s^r)^b$$

becomes large only when:

$$s \approx 0.9$$

Practical Interpretation

1. Only document pairs with high expected similarity are returned as candidates.
2. Exact Jaccard similarity is computed only for those candidates.
3. Query time is significantly reduced.
4. Recall drops substantially because true neighbors with $s < 0.9$ are filtered out.

Fallback Case (num_perm = 16)

For num_perm = 16 and threshold = 0.9, there is no integer pair (b, r) satisfying:

$$b \cdot r = 16 \quad \text{and} \quad b \geq 2$$

that yields an effective threshold close to $\tau = 0.9$.

Consequently, the library activates a fallback mechanism: it selects the closest feasible (b, r) configuration that satisfies the structural constraints, even though the resulting S-curve cannot enforce the requested threshold strictly.

This justifies the label “**fallback_params**” in the *Parameters* column of the corresponding table row (see Table 1).

LSH without Explicit Threshold (Fixed r = 2)

In this second implementation:

$$r = 2, \quad b = \frac{\text{num_perm}}{2}$$

The probability that a pair with true Jaccard similarity s becomes a candidate is:

$$P(s) = 1 - (1 - s^r)^b$$

The effective threshold (knee of the S-curve) is approximated by:

$$s_0 \approx \left(\frac{1}{b} \right)^{1/r}$$

Effective Threshold Values ($r = 2$)

num_perm	r	b	Effective threshold s_0
16	2	8	$1/\sqrt{8} \approx 0.354$
32	2	16	$1/\sqrt{16} = 0.25$
64	2	32	$1/\sqrt{32} \approx 0.177$

Why Does $r = 2$ Yield the Highest Recall?

From:

$$P(s) = 1 - (1 - s^r)^b$$

When r is small:

- s^r does not shrink dramatically
- Therefore, $P(s)$ remains relatively large even for moderate s
- More pairs become candidates

Consequences:

- Higher recall
- More candidate comparisons
- Larger query time

If r increases:

- s^r decreases sharply (for $s < 1$)
- $P(s)$ decreases
- Fewer candidates are generated
- Recall drops

Therefore, with $r = 2$ (the smallest valid value with $b \geq 2$), the LSH is the loosest configuration and produces the highest recall.

Comparative Summary

- ✓ In all experiments, **$K = 7$ nearest neighbors** are used.
- ✓ The similarity metric used throughout the experiments is the **Jaccard** similarity.
- ✓ **Both** LSH configurations use identical preprocessing, vectorization, and evaluation procedures to ensure a fair comparison.

Method	Threshold	Recall	Time
Brute Force	–	1.0	Very high
LSH ($r = 2$)	~0.17–0.35	Higher	Moderate
LSH (threshold = 0.9)	0.9	Very low	Very low

Theoretical Insight

The behavior of LSH is governed by:

$$P(s) = 1 - (1 - s^r)^b$$

- **r** controls sharpness (strictness)
- **b** controls the slope
- The effective threshold is approximated by:

$$s_0 \approx \left(\frac{1}{b}\right)^{1/r}$$

Small **r** → loose LSH → high recall

Large **r** → strict LSH → low recall

Imposing threshold = 0.9 enforces a strict S-curve, significantly reducing candidate generation.

Table 1. Performance comparison between brute-force Jaccard similarity and MinHash LSH-based K-NN classification for different numbers of permutations and threshold configurations. Reported metrics include index construction time in seconds (BuildTime), query time in seconds (QueryTime), total runtime (in seconds), recall (fraction of true K nearest neighbors retrieved), and the corresponding LSH parameters (num_perm, threshold, b, r).

Type	BuildTime	QueryTime	TotalTime	Recall	Parameters
Brute-Force-Jaccard	0.000000	182.606849	182.606849	1.000000	–
LSH-Jaccard (Perm=16)	35.966267	16.820395	52.786663	0.025207	num_perm=16, $\tau=0.9$, mode=fallback_params
LSH-Jaccard (Perm=32)	40.440236	18.087601	58.527837	0.015827	num_perm=32, $\tau=0.9$, (b,r)=(2,16)
LSH-Jaccard (Perm=64)	54.015925	23.557514	77.573439	0.014861	num_perm=64, $\tau=0.9$, (b,r)=(3,21)
LSH-Jaccard (Perm=16)	36.646482	43.539663	80.186685	0.300968	num_perm=16, r=2, b=8
LSH-Jaccard (Perm=32)	45.021401	68.034477	113.055878	0.445161	num_perm=32, r=2, b=16
LSH-Jaccard (Perm=64)	63.312638	134.708555	198.021194	0.639589	num_perm=64, r=2, b=32

Appendix A - Source Code (MinHash LSH with Fixed Threshold ($\tau = 0.9$))

```

import pandas as pd
import numpy as np
import re
import time

from sklearn.feature_extraction.text import ENGLISH_STOP_WORDS, CountVectorizer
from collections import Counter
from datasketch import MinHash, MinHashLSH

# =====
# 0) LOAD DATA
# =====

train = pd.read_excel(r"train.xlsx")
test = pd.read_excel(r"test_without_labels.xlsx")

y_train = train["Label"].astype(str).values

# =====
# 1) SAME PREPROCESSING AS Q1
# =====

BASE_STOPWORDS = set(ENGLISH_STOP_WORDS)
STOPWORDS = BASE_STOPWORDS | {
    "said", "says", "also", "new", "year", "week", "day", "mr", "mrs",
    "reuters", "cnn", "bbc", "news", "people", "company", "million",
    "percent", "world", "according", "make", "billion", "today"
}

def clean_text(s):
    if not isinstance(s, str):
        return ""
    s = s.lower()
    s = re.sub(r"http\S+|www.\S+|\S+@\S+|\d+", " ", s)
    s = re.sub(r"^[a-z\s]", " ", s)

```

```

        return re.sub(r"\s+", " ", s).strip()

def tokenize_filter(s):
    return " ".join([w for w in s.split() if w not in STOPWORDS and len(w) >= 3])

X_train_text = np.array([
    tokenize_filter(clean_text(t))
    for t in (train["Title"].fillna("") + " " + train["Content"].fillna(""))
])

X_test_text = np.array([
    tokenize_filter(clean_text(t))
    for t in (test["Title"].fillna("") + " " + test["Content"].fillna(""))
])

# =====
# 2) BINARY BoW FOR JACCARD
# =====

vec = CountVectorizer(
    lowercase=False,
    tokenizer=str.split,
    preprocessor=None,
    token_pattern=None,
    binary=True,
    min_df=2,
    max_df=0.95,
    max_features=100_000
)

X_train_bin = vec.fit_transform(X_train_text).astype(np.bool_)
X_test_bin = vec.transform(X_test_text).astype(np.bool_)
X_train_u8 = X_train_bin.astype(np.uint8)
X_test_u8 = X_test_bin.astype(np.uint8)

train_nnz = np.asarray(X_train_bin.getnnz(axis=1)).reshape(-1)

# =====
# 3) BRUTE FORCE KNN (K=7)
# =====

def brute_force_knn(Xtr, ytr, Xq, k=7):
    maj = Counter(ytr).most_common(1)[0][0]
    preds = []
    topk_indices_all = []

    inter = (Xq @ Xtr.T).tocsr()
    q_nnz = np.asarray(Xq.getnnz(axis=1)).reshape(-1)

    for i in range(inter.shape[0]):
        row = inter.getrow(i)
        if row.nnz == 0:
            preds.append(maj)
            topk_indices_all.append(set())
            continue

        idx = row.indices
        inter_counts = row.data.astype(np.float32)
        union = q_nnz[i] + train_nnz[idx] - inter_counts
        jac = inter_counts / np.maximum(union, 1)

        kk = min(k, len(jac))
        topk = idx[np.argpartition(-jac, kk - 1)[:kk]]
        preds.append(Counter(ytr[topk]).most_common(1)[0][0])
        topk_indices_all.append(set(topk))

```



```

        return np.array(preds), topk_indices_all

print("Running Brute Force...")
t0 = time.time()
bf_preds, bf_topk = brute_force_knn(X_train_u8, y_train, X_test_u8, k=7)
bf_query_time = time.time() - t0
print("Brute Force QueryTime:", bf_query_time)

# =====
# 4) LSH with explicit threshold (tau=0.9)
#   datasketch chooses (b,r) internally based on num_perm & threshold
# =====

def _closest_params_for_tau(num_perm, tau):
    """
    Find feasible (b,r) with:
    - b*r == num_perm
    - b >= 2
    whose knee  $s_0 \approx (1/b)^{(1/r)}$  is closest to tau.
    """
    best = None
    best_diff = float("inf")

    for r in range(1, num_perm + 1):
        if num_perm % r != 0:
            continue
        b = num_perm // r
        if b < 2:
            continue
        s0 = (1.0 / b) ** (1.0 / r)
        diff = abs(s0 - tau)
        if diff < best_diff:
            best_diff = diff
            best = (b, r, s0)

    if best is None:
        raise ValueError(f"No feasible (b,r) for num_perm={num_perm} with b>=2.")
    return best # (b, r, s0)

def build_lsh(X_text, num_perm=32, tau=0.9):
    """
    1) Try exact teacher-style: MinHashLSH(threshold=tau)
    2) If datasketch fails (b<2), fallback to closest feasible (b,r) with
    b*r=num_perm.
    """
    try:
        lsh = MinHashLSH(num_perm=num_perm, threshold=tau)
        mode = "threshold"
        chosen = (lsh.b, lsh.r, None) # s0 unknown here
    except ValueError as e:
        if "bands are too small" not in str(e):
            raise
        b, r, s0 = _closest_params_for_tau(num_perm=num_perm, tau=tau)
        lsh = MinHashLSH(num_perm=num_perm, params=(b, r))
        mode = "fallback_params"
        chosen = (b, r, s0)

    t0 = time.time()
    for i, doc in enumerate(X_text):
        mh = MinHash(num_perm=num_perm)
        tokens = set(doc.split())
        mh.update_batch([t.encode("utf8") for t in tokens])
        lsh.insert(str(i), mh)

    build_time = time.time() - t0
    return lsh, build_time, mode, chosen

```

```

def lsh_knn(lsh, X_test_text, y_train, bf_topk, num_perm=32, k=7):
    maj = Counter(y_train).most_common(1)[0][0]
    preds = []
    total_hits = 0
    total_possible = 0

    t0 = time.time()

    for i, doc in enumerate(X_test_text):
        mh = MinHash(num_perm=num_perm)
        tokens = set(doc.split())
        mh.update_batch([t.encode("utf8") for t in tokens])
        candidates = lsh.query(mh)

        if len(candidates) == 0:
            preds.append(maj)
            total_possible += k
            continue

        cand_idx = np.array([int(c) for c in candidates])

        # exact Jaccard only on candidates (for ranking top-k)
        q_vec = X_test_u8[i]
        inter = (q_vec @ X_train_u8[cand_idx].T).toarray().ravel()
        q_nnz = q_vec.getnnz()
        union = q_nnz + train_nnz[cand_idx] - inter
        jac = inter / np.maximum(union, 1)

        if cand_idx.size <= k:
            topk_local = cand_idx
        else:
            topk_local = cand_idx[np.argpartition(-jac, k - 1)[:k]]

        preds.append(Counter(y_train[topk_local]).most_common(1)[0][0])

        total_hits += len(set(topk_local) & bf_topk[i])
        total_possible += k

    query_time = time.time() - t0
    recall = total_hits / total_possible
    return np.array(preds), query_time, recall

# =====
# 5) RUNS (tau fixed to 0.9 as required)
# =====

TAU = 0.9
results = []

for perm in [16, 32, 64]:
    print(f"\n==== LSH num_perm={perm}, threshold={TAU} =====")

    lsh, build_time, mode, (b_used, r_used, s0_used) = build_lsh(X_train_text,
num_perm=perm, tau=TAU)

    preds, query_time, recall = lsh_knn(
        lsh, X_test_text, y_train,
        bf_topk,
        num_perm=perm,
        k=7
    )

    total_time = build_time + query_time

    results.append({

```

```

        "Type": f"LSH-Jaccard (Perm={perm})",
        "BuildTime": build_time,
        "QueryTime": query_time,
        "TotalTime": total_time,
        "Recall": recall,
        "Params": f"perm={perm}, tau={TAU}, mode={mode}, (b,r)={({b_used},{r_used})" +
(f", s0={s0_used:.3f}" if s0_used is not None else "")
    })

# Brute row
results.insert(0, {
    "Type": "Brute-Force-Jaccard",
    "BuildTime": 0.0,
    "QueryTime": bf_query_time,
    "TotalTime": bf_query_time,
    "Recall": 1.0,
    "Params": "-"
})

results_df = pd.DataFrame(results)
print(results_df)

```

Appendix B - Source Code (MinHash LSH with Fixed Rows (r = 2))

```

import pandas as pd
import numpy as np
import re
import time

from sklearn.feature_extraction.text import ENGLISH_STOP_WORDS, CountVectorizer
from collections import Counter
from scipy import sparse
from datasketch import MinHash, MinHashLSH

# =====
# 0) LOAD DATA
# =====

train = pd.read_excel(r"train.xlsx")
test = pd.read_excel(r"test_without_labels.xlsx")

y_train = train["Label"].astype(str).values

# =====
# 1) SAME PREPROCESSING AS Q1
# =====

BASE_STOPWORDS = set(ENGLISH_STOP_WORDS)
STOPWORDS = BASE_STOPWORDS | {
    "said", "says", "also", "new", "year", "week", "day", "mr", "mrs",
    "reuters", "cnn", "bbc", "news", "people", "company", "million",
    "percent", "world", "according", "make", "billion", "today"
}

def clean_text(s):
    if not isinstance(s, str):
        return ""
    s = s.lower()
    s = re.sub(r"http\S+|www.\S+|\S+@\S+|\d+", " ", s)
    s = re.sub(r"^[^a-z\s]", " ", s)
    return re.sub(r"\s+", " ", s).strip()

def tokenize_filter(s):
    return " ".join([w for w in s.split() if w not in STOPWORDS and len(w)>=3])

```

```

X_train_text = np.array([
    tokenize_filter(clean_text(t))
    for t in (train["Title"].fillna("") + " " + train["Content"].fillna(""))
])

X_test_text = np.array([
    tokenize_filter(clean_text(t))
    for t in (test["Title"].fillna("") + " " + test["Content"].fillna(""))
])

# =====
# 2) BINARY BoW FOR JACCARD
# =====

vec = CountVectorizer(
    lowercase=False,
    tokenizer=str.split,
    preprocessor=None,
    token_pattern=None,
    binary=True,
    min_df=2,
    max_df=0.95,
    max_features=100_000
)

X_train_bin = vec.fit_transform(X_train_text).astype(np.bool_)
X_test_bin = vec.transform(X_test_text).astype(np.bool_)
X_train_u8 = X_train_bin.astype(np.uint8)
X_test_u8 = X_test_bin.astype(np.uint8)

train_nnz = np.asarray(X_train_bin.getnnz(axis=1)).reshape(-1)

# =====
# 3) BRUTE FORCE KNN (K=7)
# =====

def brute_force_knn(Xtr, ytr, Xq, k=7):
    maj = Counter(ytr).most_common(1)[0][0]
    preds = []
    topk_indices_all = []

    inter = (Xq @ Xtr.T).tocsr()
    q_nnz = np.asarray(Xq.getnnz(axis=1)).reshape(-1)

    for i in range(inter.shape[0]):
        row = inter.getrow(i)
        if row.nnz == 0:
            preds.append(maj)
            topk_indices_all.append([])
            continue

        idx = row.indices
        inter_counts = row.data.astype(np.float32)
        union = q_nnz[i] + train_nnz[idx] - inter_counts
        jac = inter_counts / np.maximum(union, 1)

        topk = idx[np.argpartition(-jac, min(k, len(jac)) - 1)[:k]]
        preds.append(Counter(ytr[topk]).most_common(1)[0][0])
        topk_indices_all.append(set(topk))

    return np.array(preds), topk_indices_all

print("Running Brute Force...")
t0 = time.time()
bf_preds, bf_topk = brute_force_knn(X_train_u8, y_train, X_test_u8, k=7)

```

```

bf_query_time = time.time() - t0

print("Brute Force QueryTime:", bf_query_time)

def build_lsh(X_text, num_perm=32):

    rows = 2
    bands = num_perm // rows

    lsh = MinHashLSH(num_perm=num_perm, params=(bands, rows))

    t0 = time.time()

    for i, doc in enumerate(X_text):
        mh = MinHash(num_perm=num_perm)
        tokens = set(doc.split())
        mh.update_batch([t.encode("utf8") for t in tokens])
        lsh.insert(str(i), mh)

    build_time = time.time() - t0
    return lsh, build_time

def lsh_knn(lsh, X_test_text, y_train, bf_topk, num_perm=32, k=7):

    maj = Counter(y_train).most_common(1)[0][0]
    preds = []
    total_hits = 0
    total_possible = 0

    t0 = time.time()

    for i, doc in enumerate(X_test_text):

        mh = MinHash(num_perm=num_perm)
        tokens = set(doc.split())
        mh.update_batch([t.encode("utf8") for t in tokens])
        candidates = lsh.query(mh)

        if len(candidates)==0:
            preds.append(maj)
            total_possible += k
            continue

        cand_idx = np.array([int(c) for c in candidates])

        # exact Jaccard only on candidates
        q_vec = X_test_u8[i]
        inter = (q_vec @ X_train_u8[cand_idx].T).toarray().ravel()
        q_nnz = q_vec.getnnz()
        union = q_nnz + train_nnz[cand_idx] - inter
        jac = inter / np.maximum(union,1)
        if len(cand_idx) <= k:
            topk_local = cand_idx
        else:
            topk_local = cand_idx[np.argpartition(-jac, k-1)[:k]]

        preds.append(Counter(y_train[topk_local]).most_common(1)[0][0])

        # recall
        total_hits += len(set(topk_local) & bf_topk[i])
        total_possible += k

    query_time = time.time() - t0
    recall = total_hits / total_possible

    return np.array(preds), query_time, recall

```

```

results = []

for perm in [16,32,64]:

    print(f"\n===== LSH num_perm={perm} =====")

    lsh, build_time = build_lsh(X_train_text, num_perm=perm)

    preds, query_time, recall = lsh_knn(
        lsh, X_test_text, y_train,
        bf_topk,
        num_perm=perm
    )

    total_time = build_time + query_time

    results.append({
        "Type": f"LSH-Jaccard (Perm={perm})",
        "BuildTime": build_time,
        "QueryTime": query_time,
        "TotalTime": total_time,
        "Recall": recall
    })

# Brute row
results.insert(0,{
    "Type": "Brute-Force-Jaccard",
    "BuildTime": 0,
    "QueryTime": bf_query_time,
    "TotalTime": bf_query_time,
    "Recall": 1.0
})

results_df = pd.DataFrame(results)
print(results_df)

```

Question 3 - Dynamic Time Warping

Mathematical Formulation

Let:

- $A = (\alpha_1, \alpha_2, \dots, \alpha_n)$
- $B = (b_1, b_2, \dots, b_n)$

The local cost between two points is defined as:

$$c(i, j) = |a_i - b_j|$$

The DTW cumulative cost matrix is defined recursively as:

$$dp(i, j) = c(i, j) + \min \begin{cases} dp(i-1, j) \\ dp(i, j-1) \\ dp(i-1, j-1) \end{cases}$$

with boundary conditions:

$$dp(0, 0) = 0$$

$$dp(i, 0) = \infty \quad \text{for } i > 0$$

$$dp(0, j) = \infty \quad \text{for } j > 0$$

The DTW distance is:

$$DTW(A, B) = dp(n, m)$$

Implementation Details

1. Parsing

The dataset contains time series stored as string representations of Python lists. Each string is safely converted to a list of floats using *ast.literal_eval*.

2. Dynamic Programming Optimization

Instead of storing the full $(n + 1) \times (m + 1)$ matrix, only two rows are stored:

- *prev* → previous row
- *curr* → current row

This reduces memory complexity from:

$$O(nm)$$

to:

$$O(m)$$

Time complexity remains:

$$O(nm)$$

3. Edge Cases

- If both sequences are empty → distance = 0
- If one sequence is empty → distance = infinity

4. Execution Time

The total execution time required to compute DTW distances for all rows of the dataset was measured using:

<code>time.perf_counter()</code>

Results:

- Total rows processed: **1002**

- Total computation time: **129.139586 seconds**

This corresponds to full evaluation of the test dataset.

5. Output File

The output file *dtw.csv* was generated in CSV format with the following structure:

id, DTW distance

6. Conclusion

A fully custom implementation of Dynamic Time Warping was developed without the use of external DTW libraries.

The implementation:

- Computes DTW distances
- Uses Euclidean local cost
- Handles sequences of different lengths
- Measures total execution time
- Produces correctly formatted CSV output
- Optimizes memory usage to $O(m)$

Appendix - Source Code (Dynamic Time Warping)

```
#
# NOTE: No DTW libraries are used. This is a custom DTW implementation.
# =====

import time
import math
import ast
import pandas as pd

# -----
# 1) INPUT PATH
# -----
INPUT_XLSX = r"dtw_test.xlsx"

# Output CSV path
OUTPUT_CSV = "dtw.csv"

# -----
# 2) Helper: safe parsing of list-string
# -----
def parse_series(s):
    """
    Converts a string like "[75.125, 75.32, 75.5]" to a list of floats.
    Uses ast.literal_eval for safety (no eval).
    """
    if isinstance(s, list):
        # already list
        return [float(x) for x in s]
    if s is None or (isinstance(s, float) and math.isnan(s)):
        return []
    s = str(s).strip()
    if s == "":
        return []
```



```

try:
    arr = ast.literal_eval(s)
except Exception as e:
    raise ValueError(f"Failed to parse series: {s[:80]}...") from e

if not isinstance(arr, (list, tuple)):
    raise ValueError(f"Parsed object is not a list/tuple: {type(arr)}")
return [float(x) for x in arr]

# -----
# 3) Euclidean local cost (1D)
# -----
def euclidean_1d(a, b):
    # For scalars: sqrt((a-b)^2) == abs(a-b)
    return abs(a - b)

# -----
# 4) Handmade DTW (DP) with O(min(n,m)) memory
# -----
def dtw_distance(seq_a, seq_b):
    """
    Computes DTW distance between two 1D sequences using:
    - local cost = Euclidean distance between points
    - allowed steps: (i-1,j), (i,j-1), (i-1,j-1)
    Uses dynamic programming with two rows to reduce memory.
    """
    n = len(seq_a)
    m = len(seq_b)

    # Handle empty sequences (edge cases)
    if n == 0 and m == 0:
        return 0.0
    if n == 0:
        return float("inf")
    if m == 0:
        return float("inf")

    # Ensure we allocate the smaller dimension for memory efficiency:
    # We'll compute DP over the second axis (m) with two rows.
    prev = [float("inf")] * (m + 1)
    curr = [float("inf")] * (m + 1)

    prev[0] = float("inf")
    curr[0] = float("inf")

    prev[1:] = [float("inf")] * m

    dp00 = 0.0

    for i in range(1, n + 1):
        # Reset current row
        curr[0] = float("inf")
        for j in range(1, m + 1):
            cost = euclidean_1d(seq_a[i - 1], seq_b[j - 1])

            # Three predecessor states:
            # dp(i-1,j)    -> prev[j]
            # dp(i,j-1)    -> curr[j-1]
            # dp(i-1,j-1) -> prev[j-1], except when i=1 and j=1 where dp(0,0)=0
            if i == 1 and j == 1:
                best_prev = dp00
            else:
                best_prev = min(prev[j],          # up
                               curr[j - 1],      # left
                               prev[j - 1])      # diag
            curr[j] = cost + best_prev

    return curr[m]

```

```

        curr[j] = cost + best_prev

    # swap rows
    prev, curr = curr, prev

    # dp(n,m) is in prev[m]
    return float(prev[m])

# -----
# 5) Main: read, compute, write, time
# -----
def main():
    # Read Excel
    df = pd.read_excel(INPUT_XLSX)

    expected_cols = set(df.columns.str.lower())
    col_id = None
    col_a = None
    col_b = None
    for c in df.columns:
        cl = c.lower()
        if cl == "id":
            col_id = c
        elif cl in ("series_a", "seq_a", "a", "s1"):
            col_a = c
        elif cl in ("series_b", "seq_b", "b", "s2"):
            col_b = c

    if col_id is None or col_a is None or col_b is None:
        raise ValueError(
            f"Could not identify required columns. Found columns: {list(df.columns)}\n"
            f"Need: id, series_a, series_b (or adjust the mapping in code).")
    )

    # Start timing ONLY the DTW computations over the whole dataset
    t0 = time.perf_counter()

    out_ids = []
    out_dists = []

    for _, row in df.iterrows():
        rid = int(row[col_id])
        seq_a = parse_series(row[col_a])
        seq_b = parse_series(row[col_b])

        dist = dtw_distance(seq_a, seq_b)

        out_ids.append(rid)
        out_dists.append(dist)

    t1 = time.perf_counter()
    total_seconds = t1 - t0

    # Create output df
    out_df = pd.DataFrame({
        "id": out_ids,
        "DTW distance": out_dists
    })

    # Write CSV exactly with comma separator and header
    out_df.to_csv(OUTPUT_CSV, index=False)

    # Print timing info
    print("=====")
    print(f"Processed {len(out_df)} rows")

```

```
print(f"Output written to: {OUTPUT_CSV}")
print(f"Total DTW computation time (seconds): {total_seconds:.6f}")
print("=====")

if __name__ == "__main__":
    main()
```